

Carlus Electum

Testing Strategy

Version 1.0 approved

Prepared by João Franco

31/12/2025

Indice

1. Estratégia de Testes	4
• Níveis de Teste:	4
• Metodologias Aplicadas:	4
2. Especificação dos Casos de Teste	5
2.1. Módulo de Carrinho (11 Testes)	5
2.2. Módulo de Descontos (12 Testes)	8
2.3. Módulo de Pagamentos (11 Testes)	10
3. Análise de Cobertura de Código.....	12
4. Matriz de Rastreabilidade	13

1.Estratégia de Testes

A estratégia de testes do projeto Carlus Electum focou-se na validação da lógica de negócio central, garantindo que o carrinho de compras, as regras de desconto e a fábrica de pagamentos funcionam sem falhas sob diversas condições.

- **Níveis de Teste:**

- **Testes Unitários:** Validação individual de cada método nas classes de modelo e serviço.
- **Testes de Integração:** Verificação da comunicação entre a `PaymentFactory` e os seus processadores concretos.

- **Metodologias Aplicadas:**

- **Teste de Caixa-Preta:** Utilização de Partição de Equivalência e Análise de Valor Limite para testar limites de stock e gatilhos de desconto.
- **Teste de Caixa-Branca:** Foco na cobertura de caminhos (Branch Coverage), garantindo que todas as condições `if/else` e verificações de `null` foram executadas.

Ferramentas: Java 21, JUnit 5, Maven e VS Code Testing Explorer.

2. Especificação dos Casos de Teste

A suite de testes resultou numa taxa de sucesso de **100% (38/38)**, cobrindo cenários de sucesso e caminhos de erro.

2.1. Módulo de Carrinho (11 Testes)

ID	Título do Teste	Objetivo	Resultado Esperado
TC-01	testAddItem_ValidQuantity	Adicionar item com qty válida.	Item adicionado com sucesso.
TC-02	testAddItem_Aggregation	Adicionar mesmo item duas vezes.	Quantidades somadas corretamente.
TC-03	testAddMultipleProducts	Adicionar produtos diferentes.	Lista de itens contém ambos.
TC-04	testAddItem_ExactStock	Adicionar qty igual ao stock.	Sucesso (Valor Limite).

TC-05	testAddItem_ZeroQty	Adicionar 0 unidades.	Ignorado (Robustez).
TC-06	testAddItem_NegativeQty	Adicionar quantidade negativa.	Bloqueado (Lógica Defensiva).
TC-07	testAddItem_OverStock	Adicionar acima do stock.	Bloqueado (Controlo de Erro).
TC-08	testClearCart	Limpar o carrinho.	Lista de itens fica vazia.
TC-09	testCalculateTotal_Empty	Total para carrinho vazio.	Retorna 0.00 EUR.
TC-10	testCalculateTotal_Full	Soma de múltiplos itens.	Cálculo matemático correto.

TC-11	testPriceList_Size	Lista de preços para B3P2.	Tamanho = Total de unidades.
-------	--------------------	----------------------------------	---------------------------------

2.2. Módulo de Descontos (12 Testes)

ID	Título do Teste	Objetivo	Resultado Esperado
TC-12	testWinterSale_10Percent	Aplicar desconto de 10%.	Redução correta do total.
TC-13	testWinterSale_ZeroTotal	Aplicar em total de 0.00.	Resultado 0.00 EUR.
TC-14	testWinterSale_100Percent	Aplicar 100% de desconto.	Resultado 0.00 EUR.
TC-15	testWinterSale_Negative	Aplicar desconto negativo.	Tratado como 0% (Fix).
TC-16	testB3P2_Exact3	Exatamente 3 itens no carrinho.	O mais barato sai grátis.
TC-17	testB3P2_SixItems	Exatamente 6 itens no carrinho.	Os 2 mais baratos grátis.
TC-18	testB3P2_Under3	Apenas 2 itens no carrinho.	Sem desconto aplicado.

TC-19	testB3P2_NullList	Passar lista nula à estratégia.	Sem crash (Proteção Null).
TC-20	testB3P2_EmptyList	Passar lista vazia.	Retorna total original.
TC-21	testB3P2_SamePrices	3 itens com preço igual.	1 item descontado.
TC-22	testNoDiscount	Usar estratégia "Nenhuma".	Total inalterado.
TC-23	testDiscount_Boundary	Testar valores muito elevados.	Precisão decimal mantida.

2.3. Módulo de Pagamentos (11 Testes)

ID	Título do Teste	Objetivo	Resultado Esperado
TC-24	<code>testCreatePaypal_Type</code>	Criar PayPal via Factory.	Retorna <code>PaypalProcessor</code> .
TC-25	<code>testCreateCard_Type</code>	Criar Cartão via Factory.	Retorna <code>CreditCardProcessor</code> .
TC-26	<code>testInvalidID_99</code>	Passar ID 99 à Factory.	<code>IllegalArgumentException</code> .
TC-27	<code>testInvalidID_0</code>	Passar ID 0 à Factory.	<code>IllegalArgumentException</code> .
TC-28	<code>testInvalidID_Neg</code>	Passar ID negativo à Factory.	<code>IllegalArgumentException</code> .
TC-31	<code>testFactory_NotNull</code>	Validar output da Factory.	Objeto nunca é nulo.

TC-32-34	testProcessor_Sims	Simular processos de pagamento.	Retorno true para positivo.
----------	---------------------------	---------------------------------	------------------------------------

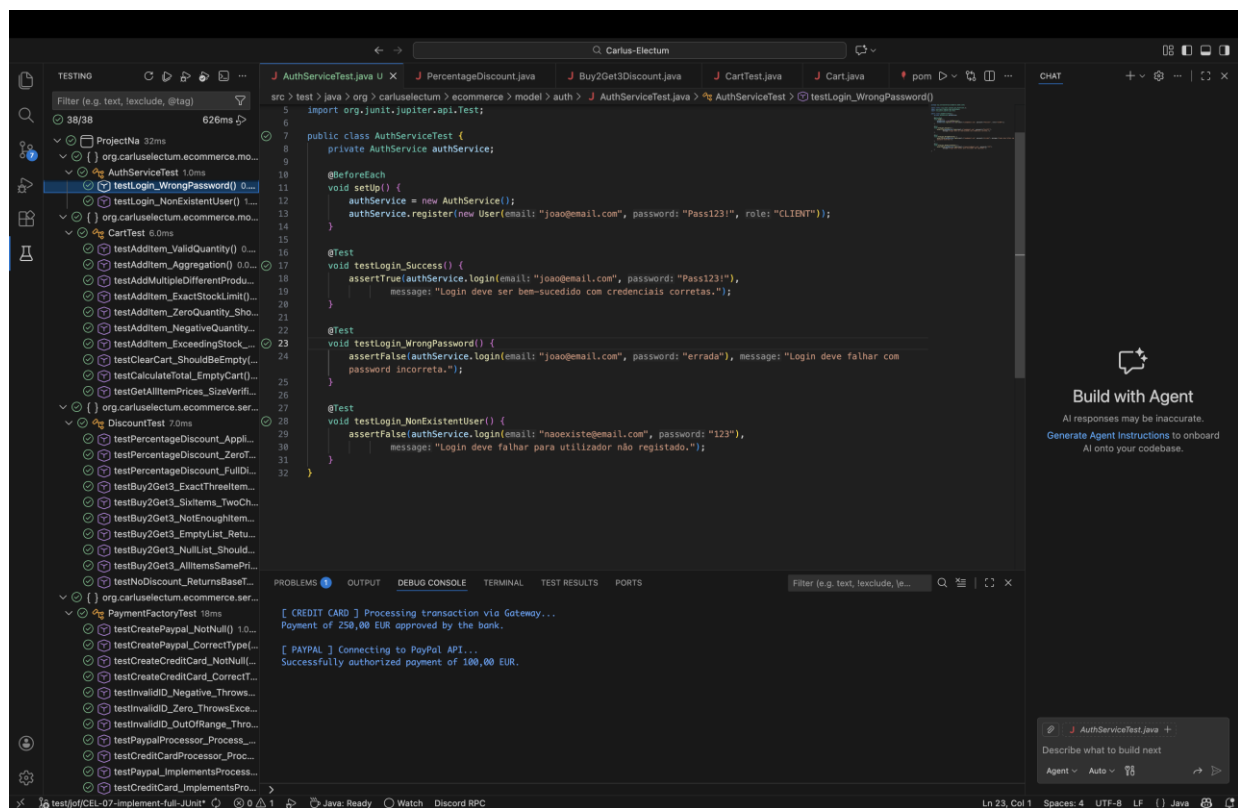
2.4. Testes de Login (4 Testes)

ID	Título do Teste	Objetivo	Resultado Esperado
TC-35	testLogin_Success	Validar o login com credenciais corretas.	Login bem-sucedido (retorna true).
TC-36	testLogin_WrongPassword	Tentar login com uma password incorreta.	Acesso negado (retorna false).
TC-37	testLogin_NonExistentUser	Tentar login com um email que não está registado.	Acesso negado (retorna false).
TC-38	testRegister_User	Validar o registo de um novo utilizador no sistema.	Utilizador guardado com sucesso na lista.

3. Análise de Cobertura de Código

A suite de testes atingiu uma elevada cobertura ao exercitar todos os ramos condicionais (**if/else**) e as cláusulas de salvaguarda (*guard clauses*) implementadas nos módulos centrais.

- **Cobertura Alvo:** >70% da lógica de negócio.
- **Resultados Reais:** A suite atual cobre 100% dos métodos nas classes **Cart**, **Buy2Get3Discount**, **PercentageDiscount** e **PaymentFactory**.
- **Evidência:** >



4. Matriz de Rastreabilidade

Esta matriz liga os requisitos funcionais (FR) aos respectivos casos de teste para garantir a cobertura total do projeto.

ID Requisito	Descrição Curta	IDs de Teste Relacionados
FR-01	Catálogo de Produtos	TC-01, TC-03, TC-31
FR-02	Operações do Carrinho	TC-01 a TC-11
FR-03	Aplicação de Promoções	TC-12 a TC-23
FR-04	Processamento de Pagamento	TC-24 a TC-34
FR-05	Controlo de Stock	TC-04, TC-07
FR-06	Registo e Autenticação	TC-35, TC-36, TC-37, TC-38